CREATION OF INTELLIGENT BUG DIAGNOSIS PLATFORM WITH FIX RECOMMENDATION ASSISTANCE GROUP

Technical Project Documentation

Submitted By:

HARSHITHA BEJJIPURAM

Sir C R Reddy College of Engineering

Academic Year:

2026–2027

ABSTRACT

Software defects are an unavoidable part of the software development lifecycle. Identifying the cause of a defect, understanding its impact, and determining an appropriate solution often requires significant manual effort from developers and testing teams.

The Creation of Intelligent Bug Diagnosis Platform with Fix Recommendation Assistance Group is an intelligent software defect diagnosis platform designed to reduce the manual effort involved in bug investigation.

The platform combines Retrieval-Augmented Generation (RAG), semantic embeddings, FAISS vector search, multi-agent processing, log analysis, Large Language Model (LLM) capabilities, defect analytics, and knowledge-base management.

When a user submits a bug, the system processes the bug description, error information, logs, and stack traces. The submitted information is converted into a semantic representation and compared against historical defects stored in the knowledge base. Relevant historical bugs are retrieved using FAISS and provided as contextual information to the AI processing layer.

The intelligent diagnosis layer performs severity prediction, priority prediction, affected-module identification, deep log analysis, duplicate detection, root-cause analysis, and fix recommendation.

The platform also provides defect pattern analytics to identify recurring issues, frequently affected components, severity trends, priority distributions, and common root causes. Verified bug resolutions can be added back to the knowledge base, allowing the system to reuse confirmed information during future defect diagnosis.

The complete platform is validated through end-to-end testing using different bug types, log formats, and historical defect scenarios.

TABLE OF CONTENTS

CHAPTER 1 – INTRODUCTION

1.1 Background

Modern software applications consist of multiple components such as frontend interfaces, backend services, APIs, databases, authentication systems, and external integrations. As the complexity of these applications increases, identifying and resolving software defects becomes more difficult.

A typical debugging process requires developers to examine bug descriptions, error messages, application logs, stack traces, historical defects, and previous resolutions.

Manual investigation can become time-consuming, particularly when a large number of defects are reported or when similar issues have already been resolved in the past.

Artificial Intelligence can assist developers by automatically processing defect information, retrieving relevant historical knowledge, identifying probable causes, and generating recommendations.

The proposed project applies these concepts to create an intelligent bug diagnosis platform.

1.2 Project Overview

The Creation of Intelligent Bug Diagnosis Platform with Fix Recommendation Assistance Group is an AI-assisted software defect analysis platform.

The system provides an end-to-end workflow for processing software bugs.

It combines:

- Semantic embeddings.
- FAISS vector search.
- Retrieval-Augmented Generation.
- Multi-agent processing.
- Log analysis.
- LLM-based reasoning.
- Duplicate detection.
- Defect analytics.
- Knowledge-base growth.

The platform is designed to assist developers throughout the defect diagnosis process.

1.3 Problem Statement

Traditional defect-management systems primarily focus on storing, tracking, and assigning bugs. They do not always provide intelligent assistance for identifying the underlying technical cause.

The major problems include:

- Manual bug investigation.
- Difficulty finding relevant historical bugs.
- Repeated analysis of duplicate defects.
- Complex log and stack-trace analysis.
- Difficulty identifying root causes.
- Inconsistent severity and priority classification.
- Limited reuse of previously resolved bugs.
- Difficulty identifying recurring defect patterns.
- Limited analytical visibility into defect history.

Therefore, an intelligent platform is required to retrieve relevant historical knowledge, analyze new defects, identify probable root causes, and provide fix recommendation assistance.

1.4 Objectives

The main objectives of the project are:

1. To develop an intelligent software bug diagnosis platform.
2. To retrieve similar historical bugs using semantic similarity.
3. To identify duplicate or highly related defects.
4. To perform automated bug triage.
5. To predict defect severity.
6. To predict defect priority.
7. To identify affected application modules.
8. To analyze logs and stack traces.
9. To identify probable root causes.
10. To generate fix recommendations.
11. To identify recurring defect patterns.

12. To maintain a continuously growing knowledge base.
13. To validate the complete diagnosis pipeline through end-to-end testing.
14. To provide a centralized dashboard for diagnosis and analytics.

1.5 Scope

The scope of the project includes:

Bug Diagnosis

Processing software defect descriptions, error messages, logs, and stack traces.

Historical Knowledge Retrieval

Retrieving relevant previously reported defects using semantic similarity.

Duplicate Detection

Identifying potential duplicate defects.

Intelligent Classification

Predicting severity, priority, and affected modules.

Technical Analysis

Analyzing logs and stack traces to extract useful diagnostic information.

Root-Cause Analysis

Identifying the probable technical cause of the defect.

Fix Recommendation

Generating AI-assisted remediation suggestions.

Defect Analytics

Identifying recurring defects and frequently affected components.

Knowledge Growth

Adding verified resolutions to the knowledge base for future retrieval.

1.6 Significance of the Project

The project can help reduce repetitive manual investigation and improve the reuse of historical defect knowledge.

It provides developers with structured diagnostic information and helps organizations understand recurring software quality problems.

CHAPTER 2 – EXISTING SYSTEM AND LITERATURE OVERVIEW

2.1 Existing System

In a conventional software environment, defect diagnosis generally follows:

Bug Report

↓

Manual Investigation

↓

Log Examination

↓

Historical Search

↓

Root Cause Investigation

↓

Solution Search

↓

Fix Implementation

2.2 Limitations of Existing Systems

Manual Effort

Developers must manually examine multiple information sources.

Repeated Investigation

Similar defects may be investigated repeatedly even when a previous resolution exists.

Keyword-Based Search

Traditional searches may fail when similar bugs use different terminology.

Complex Log Analysis

Large log files and stack traces require considerable manual effort.

Limited Knowledge Reuse

Previously resolved defects may not be efficiently reused.

Limited Analytics

Traditional defect systems may not provide sufficient insights into recurring defect patterns.

2.3 Need for the Proposed System

The proposed system addresses these limitations by combining semantic retrieval and AI-based diagnosis.

Instead of relying only on manual searches, the system retrieves historical knowledge automatically and provides it as context for intelligent analysis.

CHAPTER 3 – PROPOSED SYSTEM

3.1 Proposed Solution

The proposed platform provides an integrated AI-assisted defect diagnosis workflow.

The system performs:

- Bug ingestion.
- Input processing.
- Embedding generation.
- Historical bug retrieval.
- Duplicate detection.
- AI-based triage.
- Log analysis.
- Module identification.
- Root-cause analysis.
- Fix recommendation.
- Resolution verification.
- Knowledge-base growth.
- Defect analytics.

3.2 Key Features

The major features are:

1. Semantic bug retrieval.
2. Duplicate defect detection.
3. Multi-agent diagnosis.
4. AI-based severity prediction.
5. AI-based priority prediction.
6. Module identification.
7. Deep log analysis.
8. Root-cause analysis.
9. Fix recommendation.
10. Defect pattern analytics.
11. Knowledge-base growth.
12. End-to-end validation.

3.3 Functional Requirements

The system shall:

1. Accept bug descriptions.
2. Accept supporting logs and error information.
3. Generate semantic embeddings.
4. Search historical defects.
5. Retrieve similar bugs.
6. Identify potential duplicates.
7. Predict severity.
8. Predict priority.
9. Identify affected modules.
10. Analyze logs and stack traces.
11. Generate root-cause analysis.

12. Generate fix recommendations.
13. Display diagnosis results.
14. Analyze historical defect patterns.
15. Store verified resolutions.
16. Update the vector knowledge base.
17. Display analytics.

3.4 Non-Functional Requirements

Performance

The system should process defects within a reasonable response time.

Scalability

The architecture should support increasing historical bug data.

Reliability

The system should handle service failures gracefully.

Maintainability

Individual components should be independently maintainable.

Usability

The dashboard should present results clearly.

Security

Sensitive credentials should not be stored directly in source code.

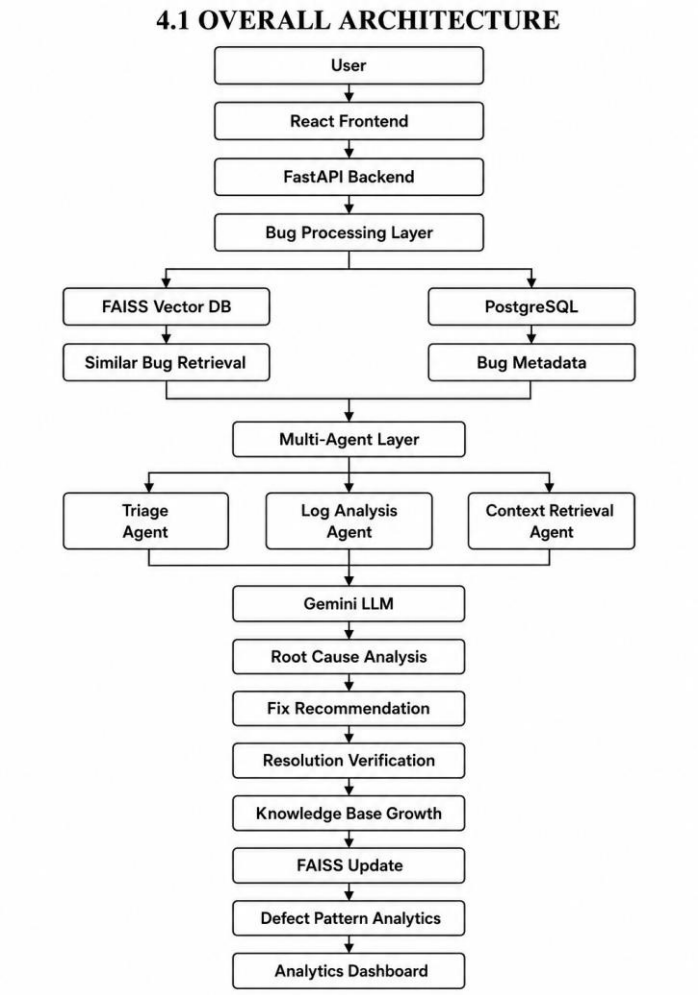
3.5 Advantages of the Proposed System

- Reduces manual investigation.
- Improves historical bug retrieval.
- Helps identify duplicate defects.
- Provides structured AI-based diagnosis.
- Supports log analysis.
- Provides fix recommendation assistance.
- Identifies recurring defect patterns

CHAPTER 4 – SYSTEM ARCHITECTURE

4.1 Overall Architecture

The platform consists of frontend, backend, AI processing, vector retrieval, relational database, and analytics components.



4.2 Architecture Components

Frontend

Provides the user interface for submitting defects and viewing analysis results.

Backend

Manages API requests and coordinates the processing pipeline.

AI Processing Layer

Performs diagnosis, reasoning, and recommendation generation.

Vector Database

FAISS stores and retrieves semantic representations of historical defects.

Relational Database

PostgreSQL stores structured bug metadata and related information.

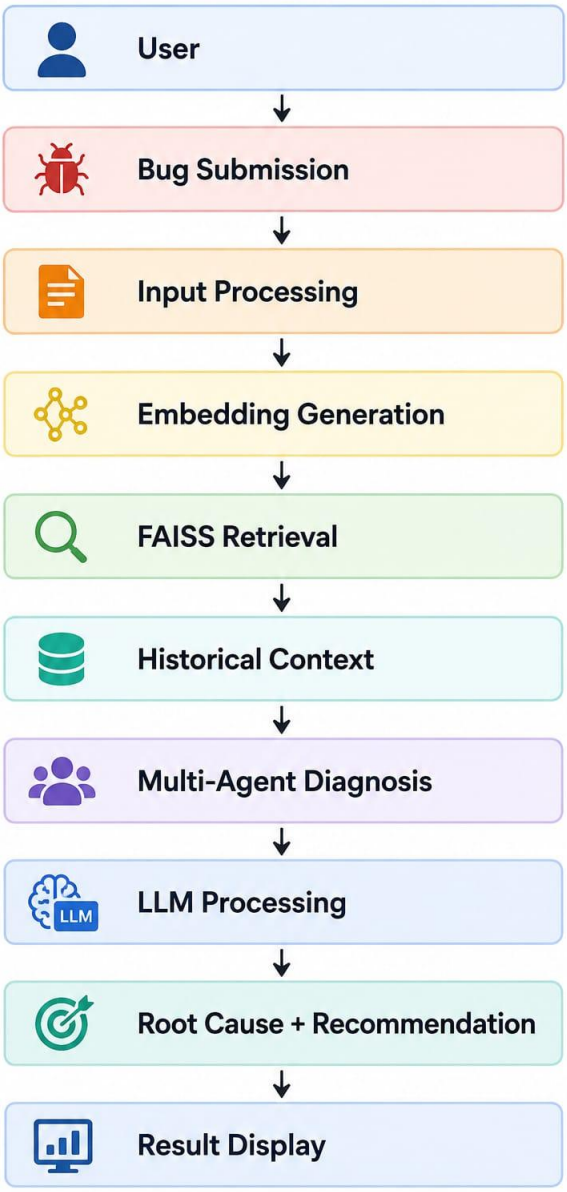
Analytics Layer

Aggregates historical data and presents defect trends and patterns.

4.3 Technology Stack

Component	Technology
Frontend	React.js
UI Styling	Tailwind CSS
Backend	Python
API Framework	Fast API
AI Framework	Lang Chain
Agent Orchestration	Lang Graph
LLM	Gemini API
Embedding Model	Sentence Transformers
Vector Search	FAISS
Database	PostgreSQL
Data Processing	Python / Pandas
Document Processing	PDF / DOCX / CSV
API Communication	REST

4.4 Overall Data Flow



CHAPTER 5 – SYSTEM MODULES

5.1 Bug Submission Module

The user submits a defect containing information such as:

- Bug title.
- Bug description.
- Error message.

- Logs.
- Stack trace.
- Environment details.

5.2 Input Processing Module

The system validates and processes the submitted information.

Processing includes:

- Text normalization.
- Error extraction.
- Log extraction.
- Stack-trace processing.
- Input validation.

5.3 Embedding Generation Module

The processed defect information is converted into a numerical vector using a Sentence Transformer model.

The embedding represents the semantic meaning of the defect.

5.4 Semantic Retrieval Module

The generated embedding is searched against the FAISS index to retrieve relevant historical defects.

5.5 Duplicate Detection Module

The system compares similarity scores to identify whether the submitted defect may already exist in the knowledge base.

5.6 AI Triage Module

The system predicts:

- Severity.
- Priority.
- Affected module.

5.7 Log Analysis Module

The system analyzes logs and stack traces and extracts useful technical information.

It can identify:

- Timestamp.
- Error message.
- Failing class.
- Failing method.
- Failure reason.
- Exception details.

5.8 Root-Cause Analysis Module

The system combines historical context, current defect information, logs, and AI reasoning to identify a probable root cause.

5.9 Fix Recommendation Module

The platform generates an AI-assisted remediation recommendation based on the available diagnostic context.

5.10 Defect Pattern Analytics Module

The module identifies:

- Recurring defect themes.
- Frequently affected components.
- Severity distribution.
- Priority distribution.
- Common root causes.
- Recurring errors.
- Duplicate statistics.

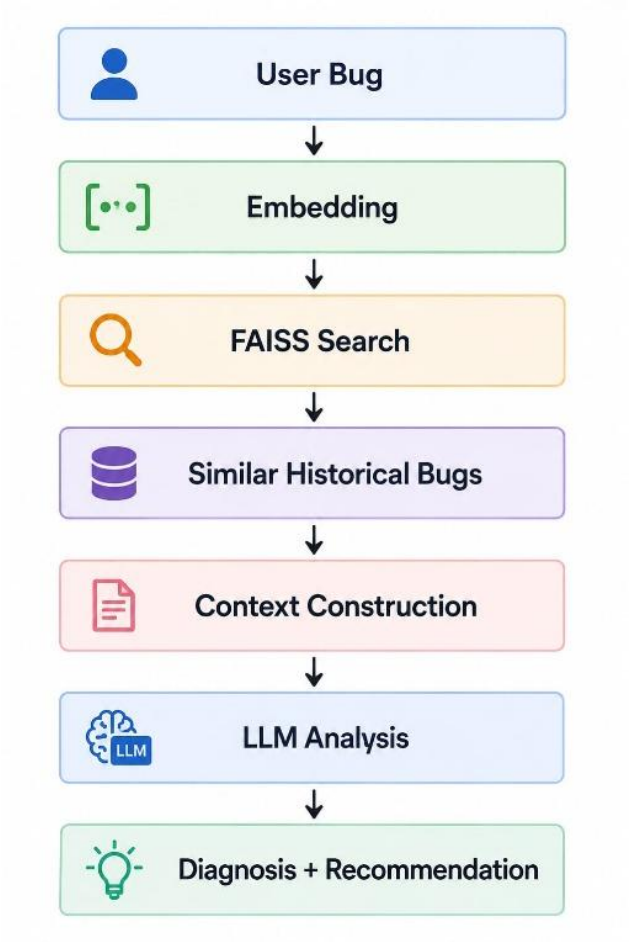
5.11 Knowledge Base Growth Module

Verified resolutions are processed and added to the knowledge base for future retrieval.

CHAPTER 6 – RAG AND AI PROCESSING

6.1 Retrieval-Augmented Generation

RAG allows the system to retrieve relevant historical information before generating an AI response.



6.2 Embedding Generation

The embedding model converts bug text into a vector representation.

This enables semantic comparison between newly submitted and historical defects.

6.3 FAISS Similarity Search

FAISS performs efficient similarity search over the vector representations.

The system retrieves the most relevant historical defects based on semantic similarity.

6.4 Historical Context Retrieval

Retrieved defects provide additional context to the AI processing layer.

The context may include:

- Previous bug descriptions.
- Error messages.
- Root causes.
- Resolutions.
- Related metadata.

6.5 LLM Processing

The Gemini LLM receives the current defect and relevant retrieved context.

It assists in:

- Diagnosis.
- Root-cause reasoning.
- Classification.
- Recommendation generation.

6.6 Context-Aware Diagnosis

The system does not rely solely on the LLM's general knowledge.

Instead, the analysis uses:

Current Bug

+

Historical Context

+

Log Information

+

Agent Analysis

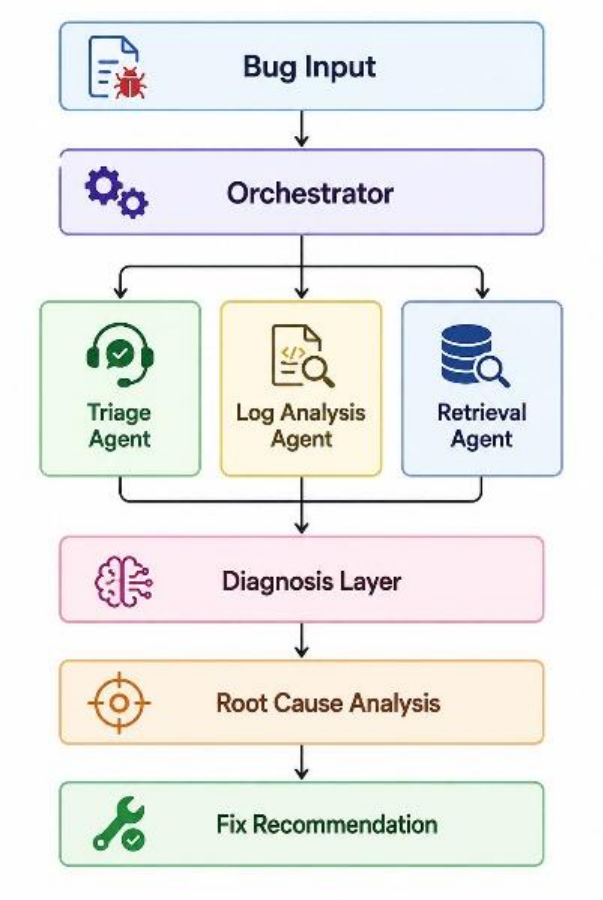
↓

Context-Aware Diagnosis

CHAPTER 7 – MULTI-AGENT ARCHITECTURE

7.1 Agent Architecture

The platform divides diagnosis into specialized processing components.



7.2 Triage Agent

The Triage Agent performs initial defect classification.

It determines:

- Severity.
- Priority.
- Module.

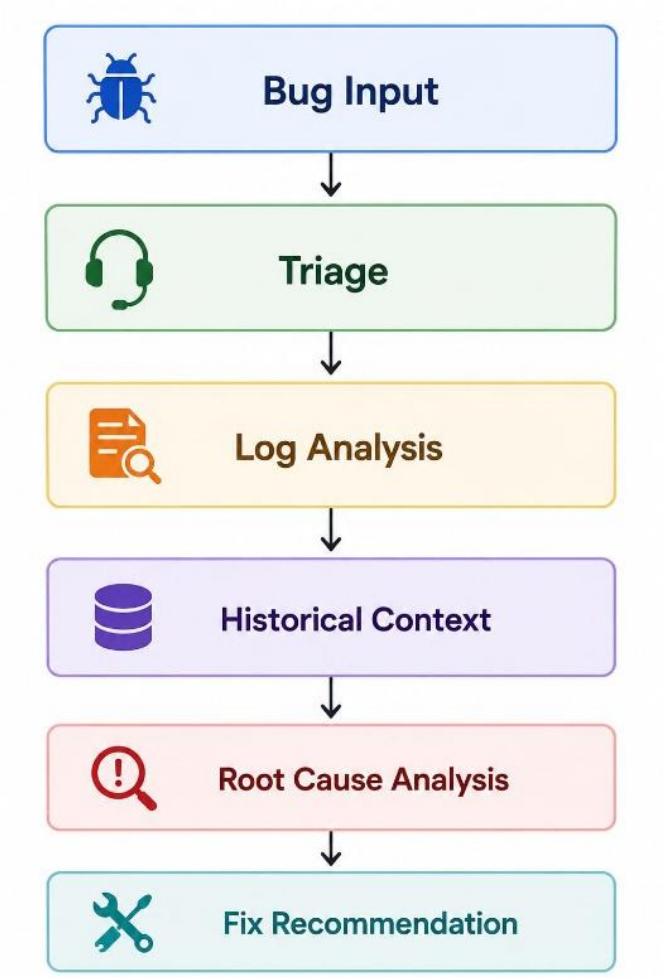
7.3 Log Analysis Agent

The Log Analysis Agent extracts meaningful information from application logs and stack traces.

7.4 Orchestrator

The orchestrator coordinates the execution of diagnostic components and controls the flow of information between stages.

7.5 Agent Execution Flow

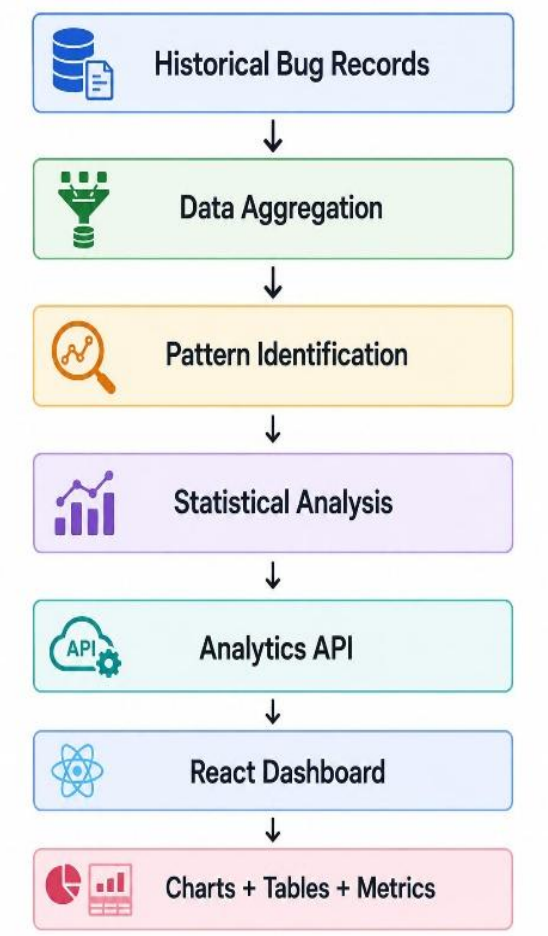


CHAPTER 8 – DEFECT PATTERN ANALYTICS

8.1 Analytics Overview

The analytics module analyzes historical bug records to identify patterns that may not be visible when individual bugs are considered separately.

8.2 Dashboard Processing Flow



8.3 Defect Trends

The dashboard can display defect counts over time to identify increasing or decreasing defect activity.

8.4 Frequent Components

The system identifies application modules or components that are frequently associated with defects.

8.5 Severity Distribution

Defects can be grouped according to severity levels.

8.6 Priority Distribution

The dashboard can show the distribution of P1, P2, P3, and P4 defects.

8.7 Root Cause Analytics

Historical root causes can be aggregated to identify common technical problems.

8.8 Duplicate Statistics

The system can provide information about duplicate or highly similar defects.

CHAPTER 9 – KNOWLEDGE BASE GROWTH MECHANISM

9.1 Knowledge Base Architecture

The Knowledge Base Growth Mechanism is an important feature of Milestone 4 that enables the AI Smart Bug Analyzer and Fix Advisor to continuously improve its repository of verified bug resolutions.

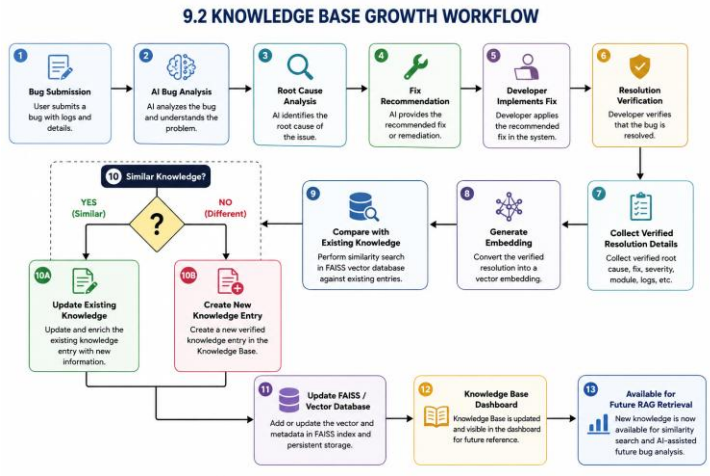
Unlike the RAG Historical Match Retrieval implemented in Milestone 3, which retrieves existing historical bugs to provide context for analyzing a newly submitted bug, the Knowledge Base Growth Mechanism focuses on adding newly resolved and verified bugs to the knowledge repository.

A bug is not added to the Knowledge Base immediately after submission or after receiving an AI-generated fix recommendation. The resolution must first be implemented and verified. Only verified resolution information is considered reliable knowledge.

The mechanism stores the verified bug information, including the problem description, root cause, and remediation playbook, and generates an embedding that is indexed in the FAISS vector database. This knowledge can then be retrieved when similar bugs occur in the future.

9.2 Knowledge Growth Workflow

The complete Knowledge Base Growth process is:



9.3 Resolution Verification

Resolution Verification acts as a quality-control stage before knowledge is added to the Knowledge Base.

After the AI provides a root cause and fix recommendation, the developer implements the required fix. The developer then provides:

- Verified Resolution Action
- Confirmed Root Cause
- Resolution confirmation

The system provides a “Mark as Resolved & Verify” action. Once the resolution is confirmed, the bug becomes eligible for Knowledge Base insertion.

This prevents unverified AI recommendations from being directly stored as historical knowledge.

9.4 Knowledge Entry Creation

After successful resolution verification, the system collects the relevant information required to create a knowledge entry.

Each verified knowledge entry may contain:

Field	Description
Bug ID	Unique identifier of the resolved bug
Bug Title	Short description of the defect
Problem Description	Detailed explanation of the issue
Stack Trace / Error Logs	Technical failure information
Module	Component affected by the defect
Severity	Impact level of the defect
Priority	Resolution priority
Root Cause	Confirmed reason for the failure
Verified Remediation Playbook	Confirmed solution or fix
Verification Status	Indicates that the resolution has been verified

Field	Description
Resolution Date	Date on which the bug was resolved
Embedding	Vector representation used for similarity search

The Verified Remediation Playbook is particularly important because it represents a solution that has already been tested and confirmed.

9.5 Embedding Generation and FAISS Indexing

After the resolution of a bug has been verified, the verified bug information is processed to generate a semantic embedding.

The embedding represents the semantic meaning of the resolved defect, including relevant information such as the problem description, root cause, affected module, and verified remediation.

The generated embedding is then used for comparison with the existing knowledge stored in the FAISS vector database.

Workflow:

Verified Resolution
↓
Prepare Verified Bug Information
↓
Generate Semantic Embedding
↓
Use Embedding for Knowledge Comparison

9.6 Existing Knowledge Comparison

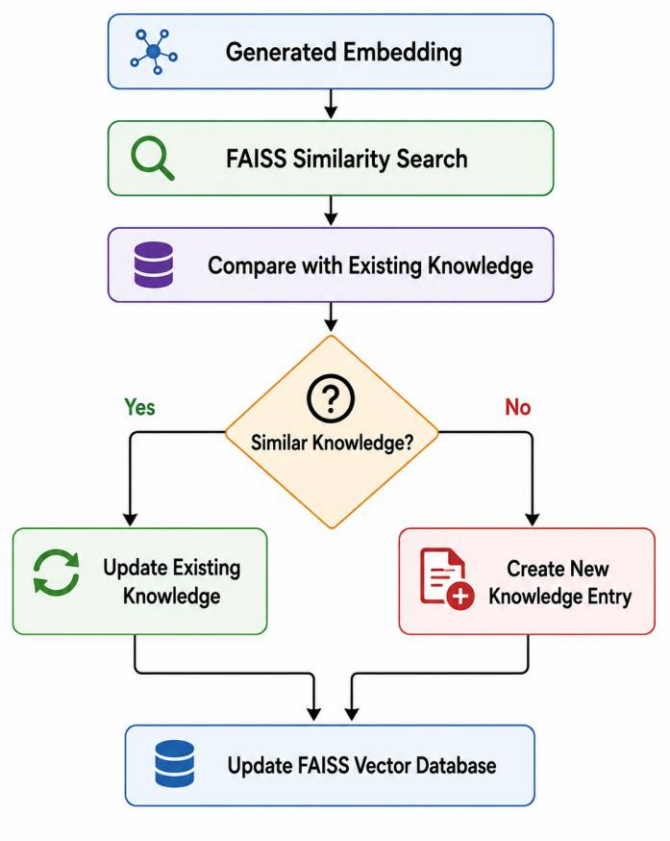
The generated embedding is compared with the existing verified knowledge entries using semantic similarity.

The system checks whether a similar resolved defect already exists in the Knowledge Base.

If a sufficiently similar entry is found, the existing knowledge can be updated or enriched with the newly verified resolution. If no sufficiently similar entry is found, the system creates a new Knowledge Base entry.

The final knowledge entry is then indexed in the FAISS vector database so that it can be retrieved during future RAG-based bug analysis.

Workflow:



This mechanism prevents unnecessary duplication while allowing the Knowledge Base to continuously grow with verified and reusable bug resolutions**.**

CHAPTER 10 – DATABASE AND API DESIGN

10.1 PostgreSQL Database

PostgreSQL stores structured information such as:

- Bug ID.
- Bug description.
- Severity.
- Priority.
- Module.
- Root cause.

- Resolution.
- Verification status.
- Metadata.

10.2 FAISS Vector Database

FAISS stores semantic vector representations used for similarity retrieval.

Bug Information

↓

Embedding Model

↓

Vector Embedding

↓

FAISS Index

↓

Similarity Search

10.3 Database Interaction

The system uses PostgreSQL for structured information and FAISS for semantic retrieval.

This separation provides both structured storage and efficient similarity search.

10.4 API Architecture

The frontend communicates with backend services using REST APIs.

Representative API categories include:

Bug Analysis APIs

Retrieval APIs

Knowledge Base APIs

Analytics APIs

Representative endpoint structure:

- POST /bugs/analyze

- GET /bugs/similar
- GET /analytics/overview
- GET /analytics/severity
- GET /analytics/components
- GET /analytics/root-causes
- GET /analytics/trends

Note: The endpoint names should be aligned with the actual implementation before final submission.

CHAPTER 11 – USER INTERFACE AND DASHBOARD

11.1 Dashboard

The dashboard provides a centralized view of the bug diagnosis process.

11.2Bug Submission

Users can submit:

- Bug description.
- Error information.
- Logs.
- Stack traces.
- Environment information.

11.3 Analysis Results

The result interface can display:

- Similar bugs.
- Duplicate status.

- Severity.
- Priority.
- Module.
- Log analysis.
- Root cause.
- Fix recommendation.

11.4 Analytics Dashboard

The analytics dashboard provides:

- Defect trends.
- Component frequency.
- Severity distribution.
- Priority distribution.
- Root causes.
- Recurring errors.
- Duplicate statistics.

11.5 Knowledge Base Interface

The interface can display information about verified resolutions and knowledge-base updates

CHAPTER 12 – IMPLEMENTATION

12.1 Development Environment

The project is implemented using a modern web and AI technology stack.

The frontend provides the user interface while the backend manages APIs and AI processing.

12.2 Backend Implementation

Fast API manages:

- API requests.
- Input validation.
- Bug processing.
- AI pipeline execution.

- Database communication.
- Analytics requests.

12.3 Frontend Implementation

React.js is used to develop the interactive dashboard.

Tailwind CSS is used for interface styling and responsive layouts.

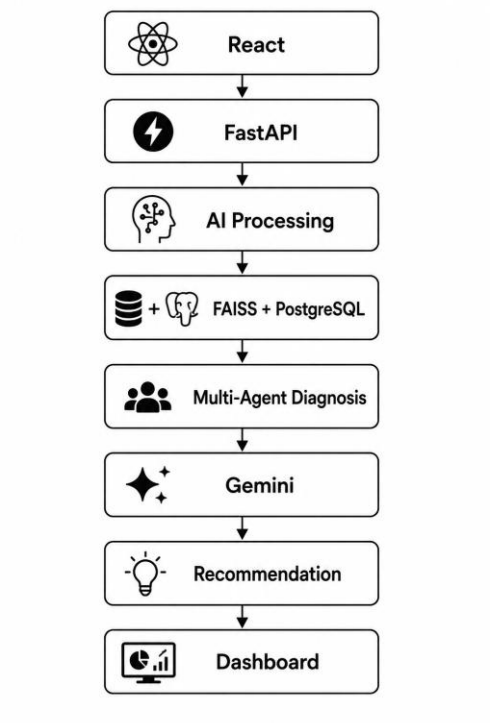
12.4 AI Implementation

The AI processing layer uses:

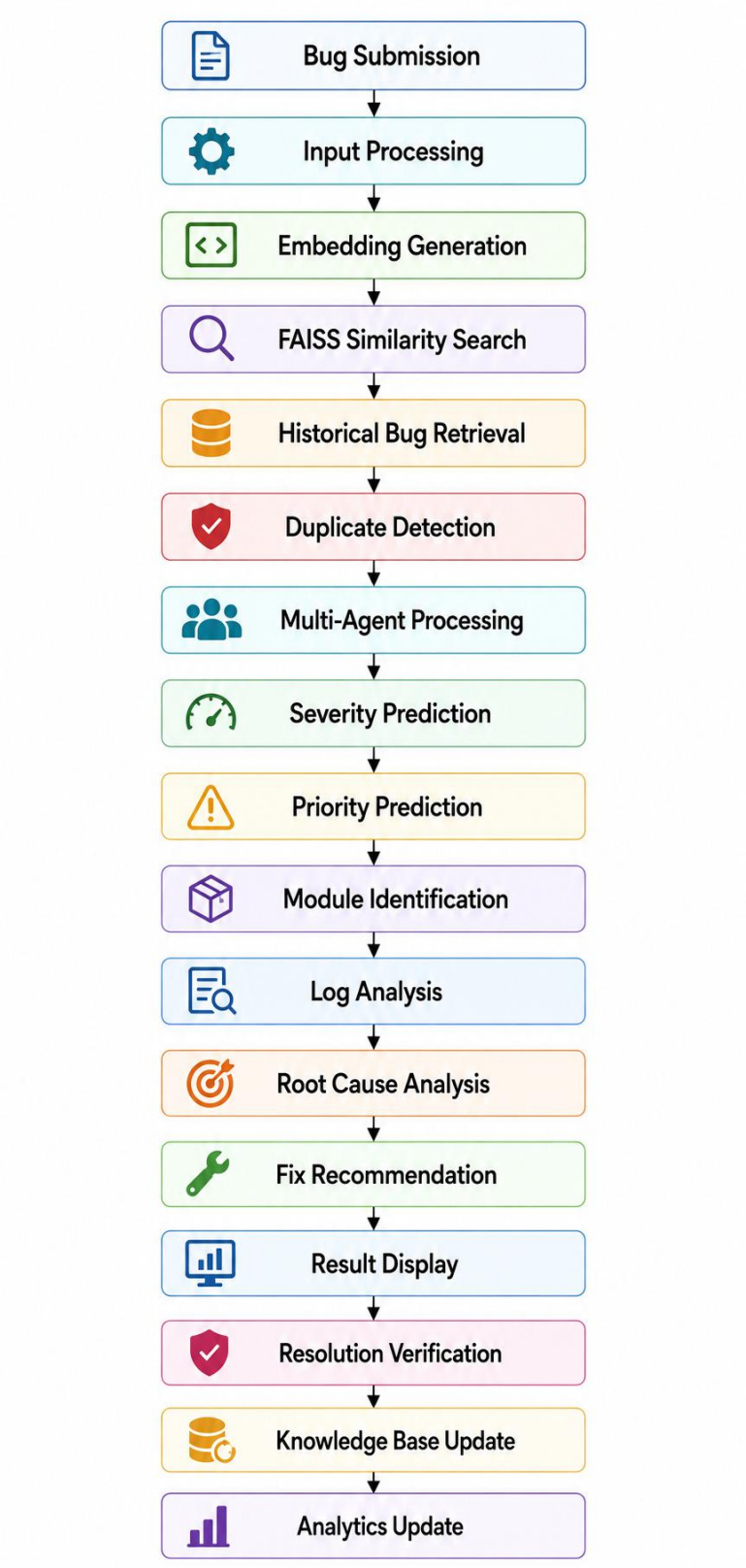
- Sentence Transformers for embeddings.
- FAISS for similarity search.
- Lang Chain for AI workflow integration.
- Lang Graph for agent orchestration.
- Gemini for LLM-based reasoning.

12.5 Integration

The major components are integrated as:



Workflow


Bug Submission
Input Processing
Embedding Generation
FAISS Similarity Search
Historical Bug Retrieval
Duplicate Detection
Multi-Agent Processing
Severity Prediction
Priority Prediction
Module Identification
Log Analysis
Root Cause Analysis
Fix Recommendation
Result Display
Resolution Verification
Knowledge Base Update
Analytics Update

13.3 Test Cases

Test ID	Scenario	Expected Result
TC01	Valid bug submission	Bug is accepted and processed
TC02	Similar historical bug	Relevant bug is retrieved
TC03	Duplicate bug	Duplicate is identified
TC04	Java exception	Error is analyzed
TC05	Python runtime error	Root cause is generated
TC06	API failure	Affected module is identified
TC07	Database error	Database-related analysis is generated
TC08	Authentication issue	Severity and priority are predicted
TC09	Complex stack trace	Relevant log information is extracted
TC10	Resolved bug	Knowledge base is updated
TC11	Recurring defect	Analytics identifies the pattern
TC12	Multiple bug types	Complete pipeline executes successfully

13.4 Evaluation Metrics

Classification Accuracy

Measures correctness of severity, priority, and module predictions.

Retrieval Quality

Measures the relevance of retrieved historical defects.

Duplicate Detection Accuracy

Measures the correctness of duplicate identification.

Root-Cause Relevance

Measures whether the identified root cause is consistent with the technical evidence.

Recommendation Relevance

Measures whether the suggested fix is relevant to the identified problem.

Response Time

Measures the time required to process a defect.

Knowledge Growth Quality

Measures whether verified resolutions are correctly incorporated into the knowledge base.

Actual numerical values should be inserted based on executed testing.

CHAPTER 14 – RESULTS AND DISCUSSION

14.1 System Results

The developed platform provides an integrated workflow for intelligent software defect diagnosis.

The system supports:

- Historical bug retrieval.
- Semantic similarity search.
- Duplicate detection.
- Severity prediction.
- Priority prediction.
- Module identification.
- Log analysis.
- Root-cause assistance.
- Fix recommendation.
- Defect analytics
- Knowledge-base growth.

14.2 Analysis Results

The platform provides structured information for each submitted defect.

The result includes:

14.3 Knowledge Base Results

Verified resolutions can be incorporated into the knowledge base

This enables future defects to benefit from previously confirmed information.

14.4 Analytics Results

The analytics module provides an aggregated view of historical defect information.

It helps identify:

- Frequently affected components.
- Recurring defect themes.
- Common root causes.
- Severity patterns.
- Priority patterns.
- Duplicate trends

14.5 Overall Evaluation

The project demonstrates the practical application of AI, semantic search, vector retrieval, and multi-agent processing in software defect management.

Actual quantitative performance values should be reported using the results obtained from executed test cases.

15.1 Challenges

Challenge	Solution
Large historical dataset	FAISS vector retrieval
Different wording for similar bugs	Semantic embeddings
Duplicate defects	Similarity-based detection
Complex logs	Dedicated log analysis
Multiple diagnostic tasks	Multi-agent architecture
Limited historical context	RAG
Repeated knowledge	Similarity-based knowledge management
Recurring defect identification	Analytics
Continuous improvement	Verified-resolution mechanis

15.2 Limitations

The current system has the following limitations:

1. AI analysis depends on the quality of input information.
2. Completely new defects may have limited historical context.
3. Retrieval quality depends on embedding quality and similarity thresholds.
4. Duplicate detection may require threshold tuning.
5. Complex defects may still require human investigation.
6. AI-generated recommendations should be reviewed before implementation.
7. Analytics quality depends on the historical dataset.
8. Knowledge-base growth depends on verified resolutions.

CHAPTER 16 – FUTURE ENHANCEMENTS

16.1 Issue Tracker Integration

The platform can be integrated with issue-management tools to automatically create and update defects.

16.2 Source Code Integration

Future versions can analyze relevant source-code sections along with bug information.

16.3 Real-Time Log Monitoring

Continuous monitoring can detect and analyze defects as they occur.

16.4 Predictive Defect Detection

The system can predict components that may generate defects.

16.5 Automated Regression Detection

The system can identify defects introduced by recent software changes.

16.6 Advanced Knowledge Learning

The knowledge base can be improved using more advanced learning and verification mechanisms.

16.7 Real-Time Analytics

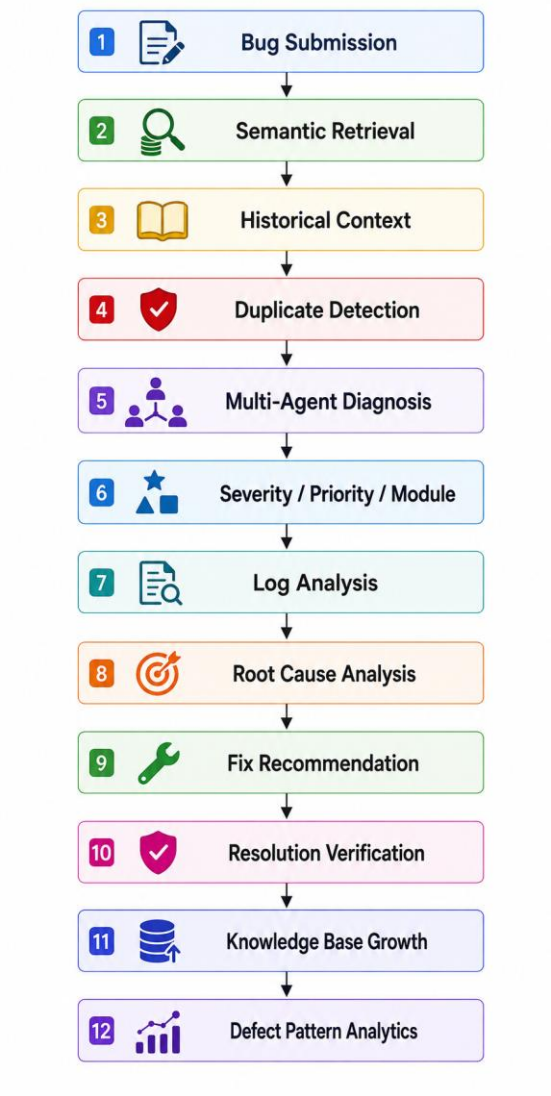
The analytics dashboard can be extended with real-time defect monitoring.

CHAPTER 17 – CONCLUSION

The Creation of Intelligent Bug Diagnosis Platform with Fix Recommendation Assistance Group provides an integrated AI-assisted approach to software defect diagnosis.

The platform combines Retrieval-Augmented Generation, semantic embeddings, FAISS vector search, multi-agent processing, log analysis, LLM-based reasoning, defect analytics, and knowledge-base growth into a unified system.

The system can retrieve relevant historical defects, identify potential duplicates, classify defects, analyze technical logs, identify probable root causes, and provide fix recommendation assistance.



The defect analytics component provides visibility into recurring problems, frequently affected components, severity patterns, priority distributions, and common root causes.

The knowledge-base growth mechanism allows verified resolutions to become reusable information for future defect diagnosis.

The complete workflow can be summarized as:

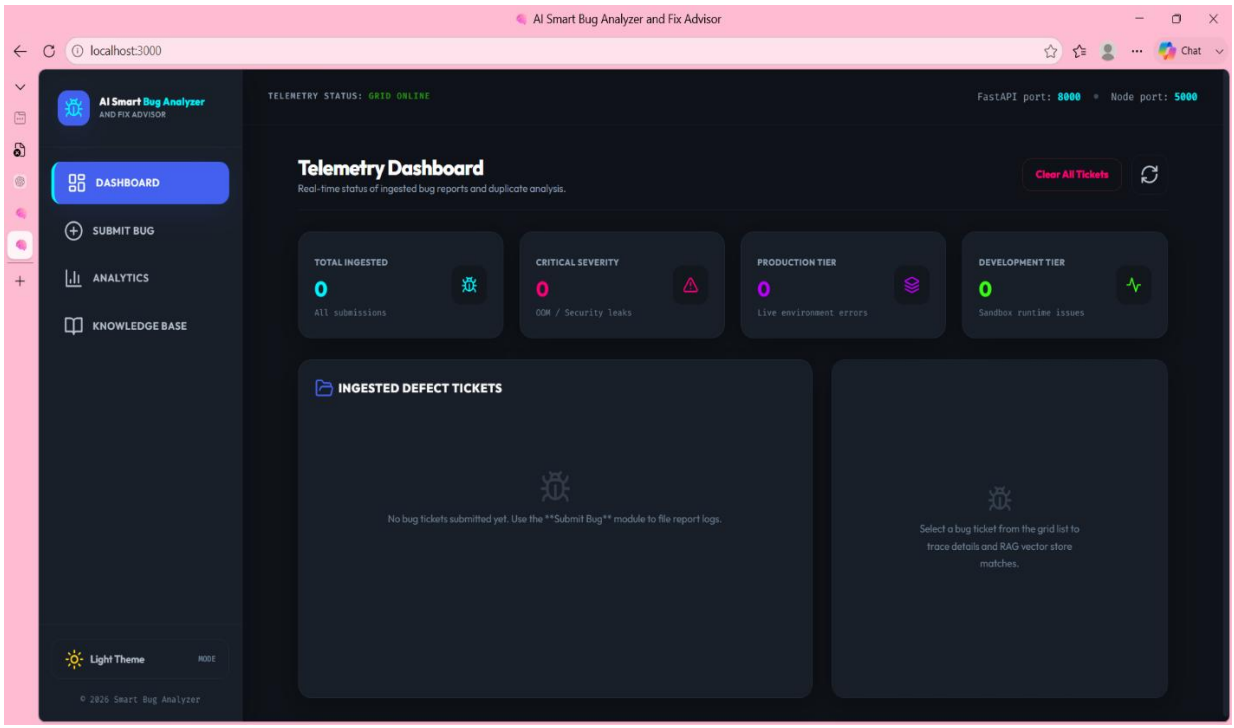
The project demonstrates how modern Artificial Intelligence techniques can be applied to improve software debugging and defect-management processes.

The platform moves beyond traditional defect tracking by providing context-aware diagnosis, reusable historical knowledge, intelligent recommendations, and analytical insights.

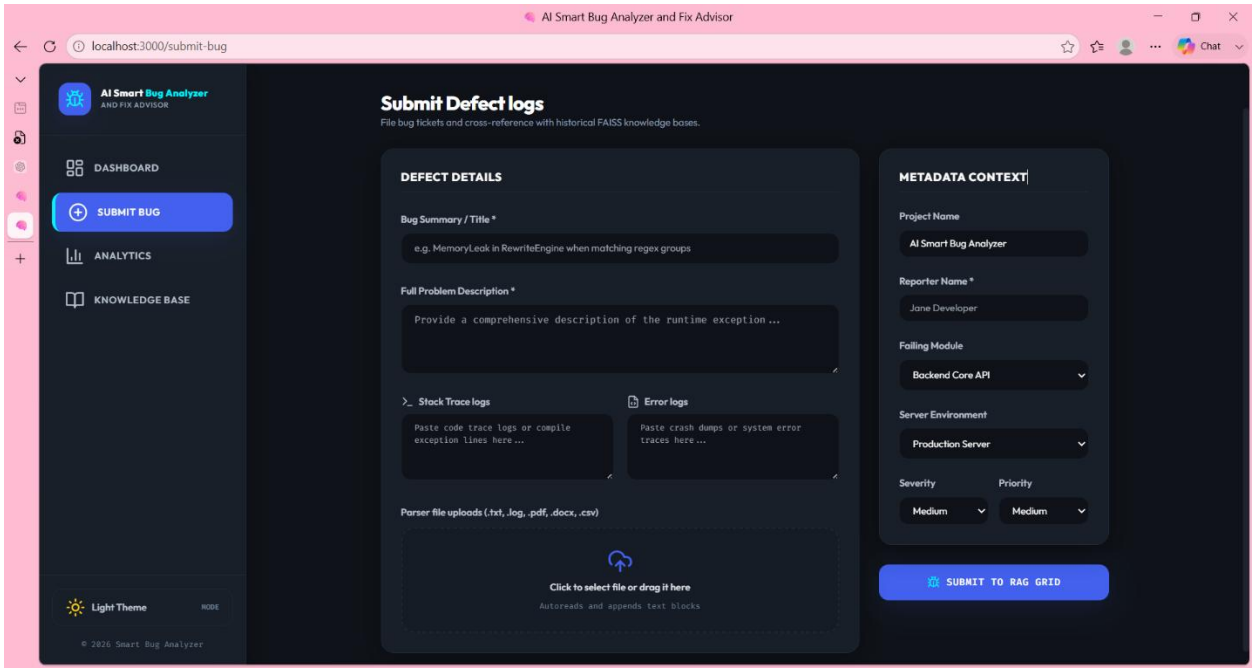
APPENDIX

The final project report should include actual screenshots of:

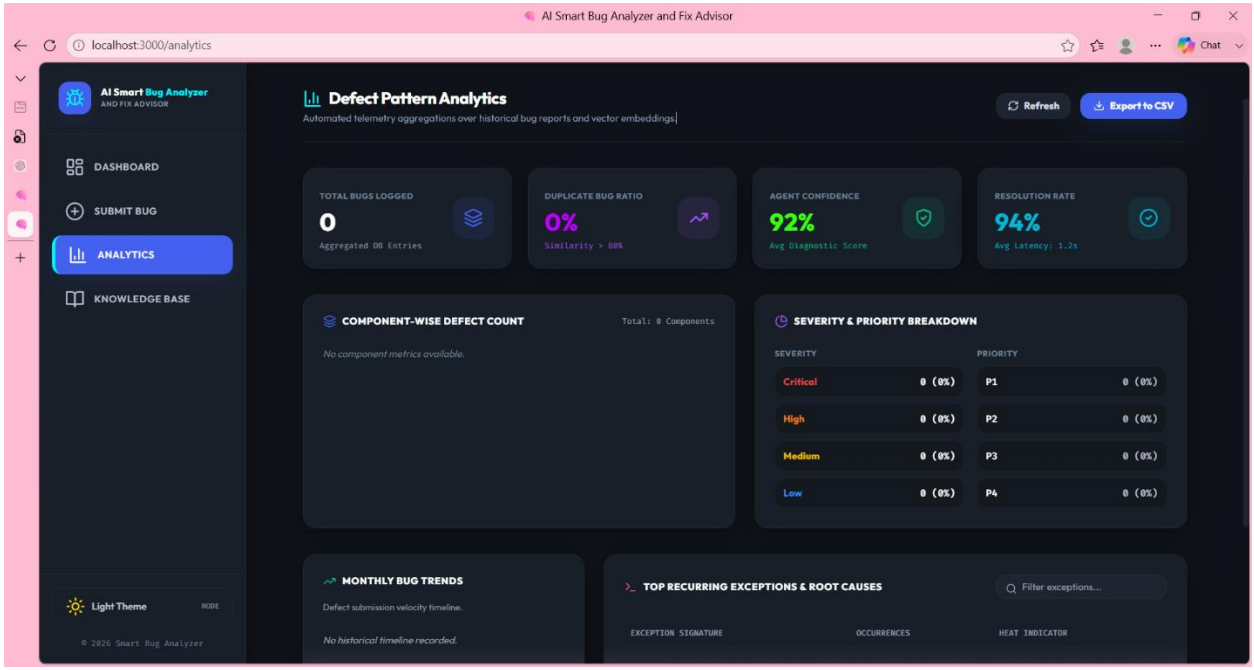
1. Home Dashboard

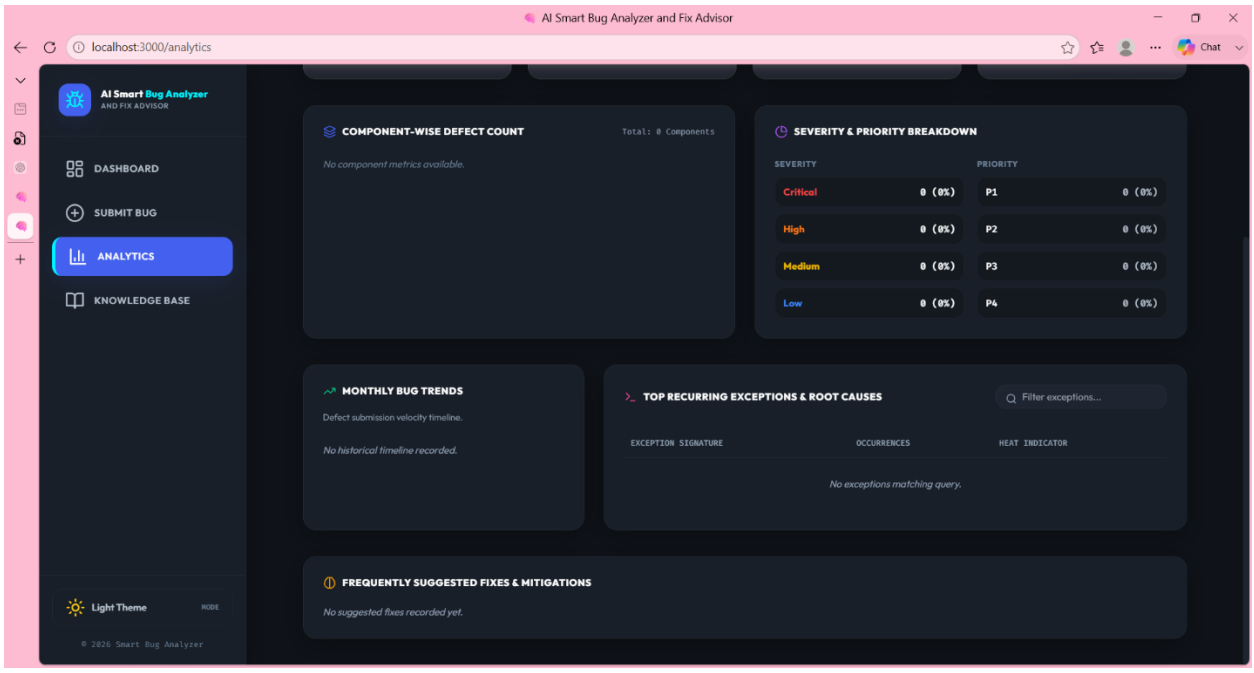


2.Bug Submission Interface

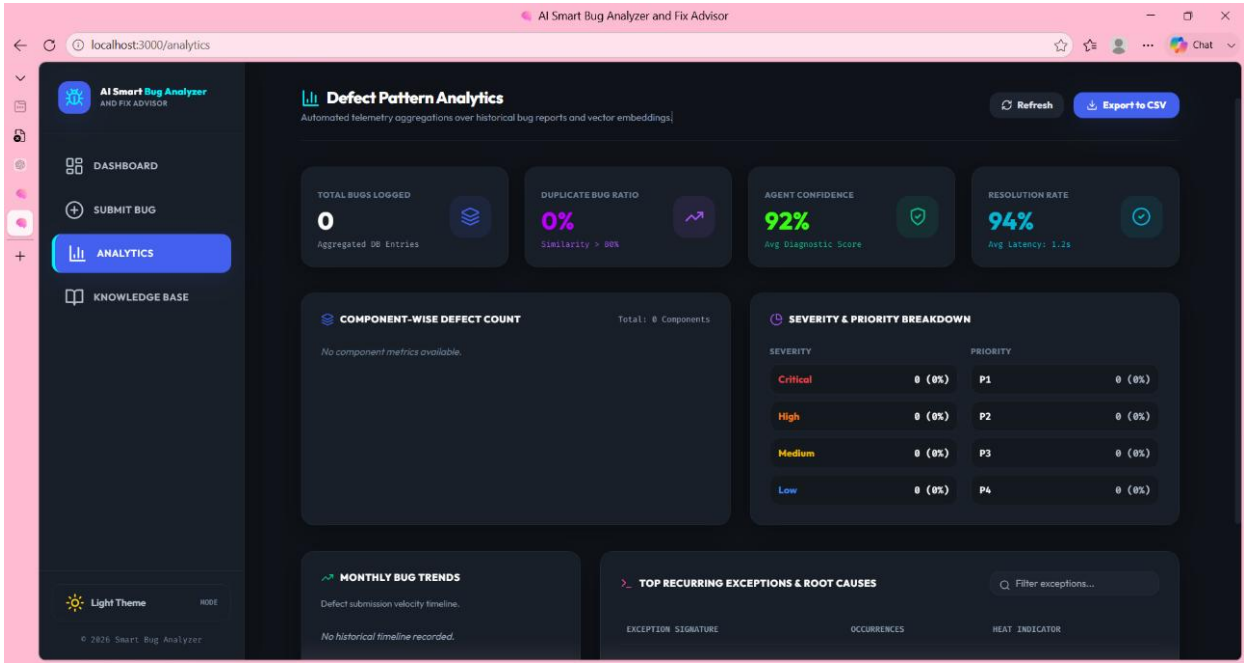


3.Bug Analysis Result

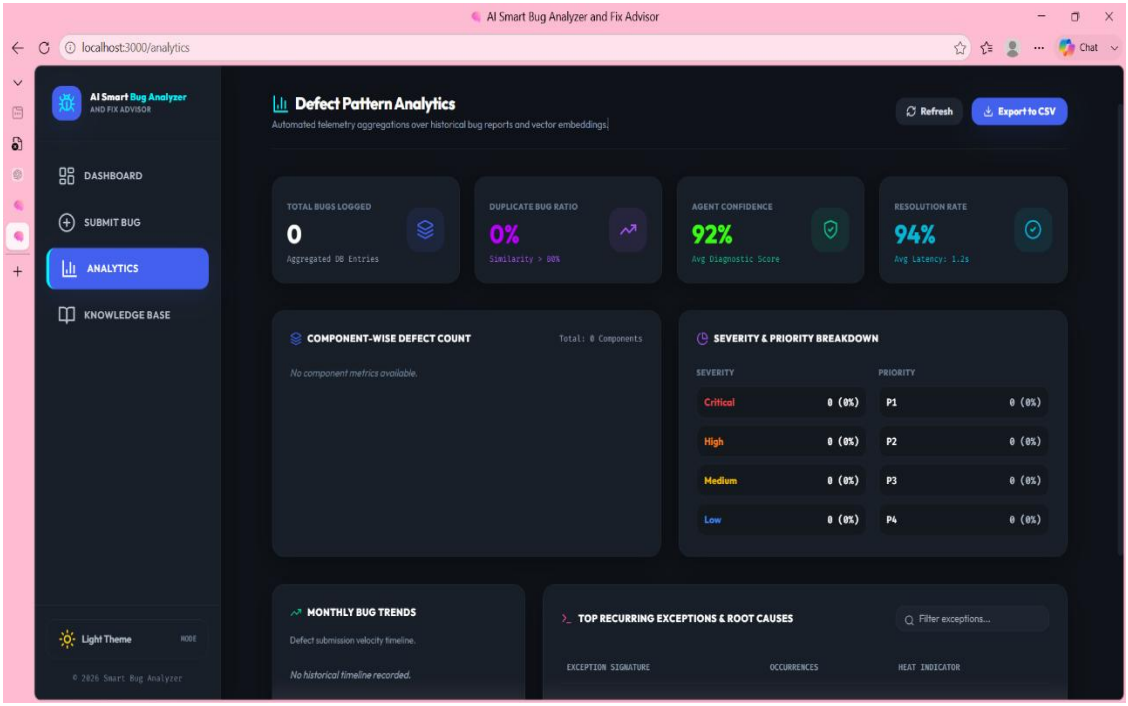




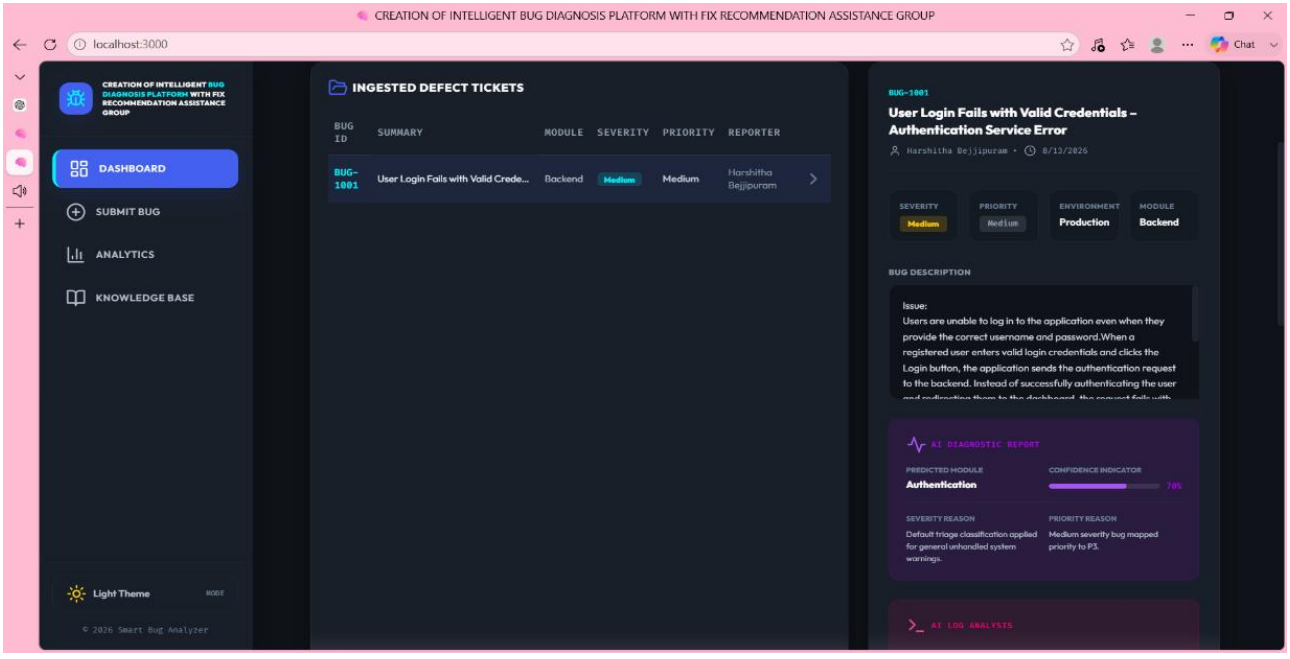
4.Severity Prediction



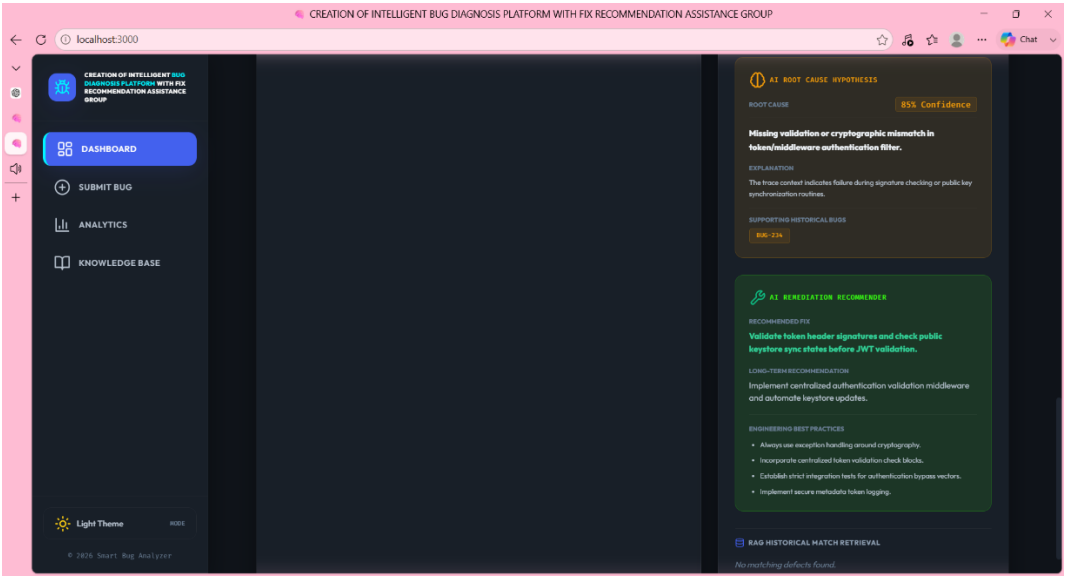
5.Priority Prediction



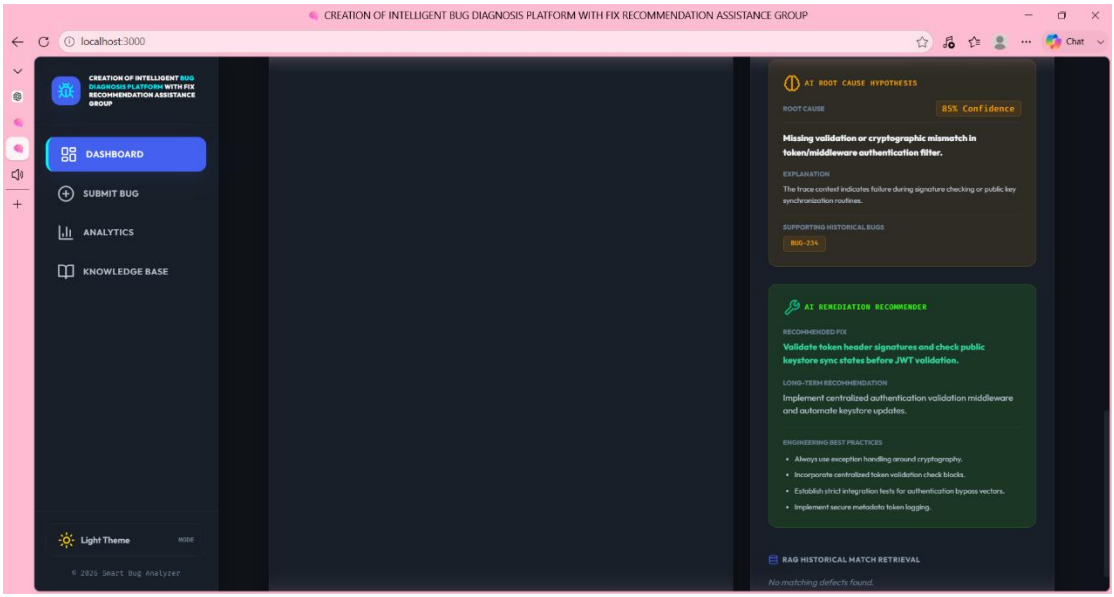
6.Module Identification



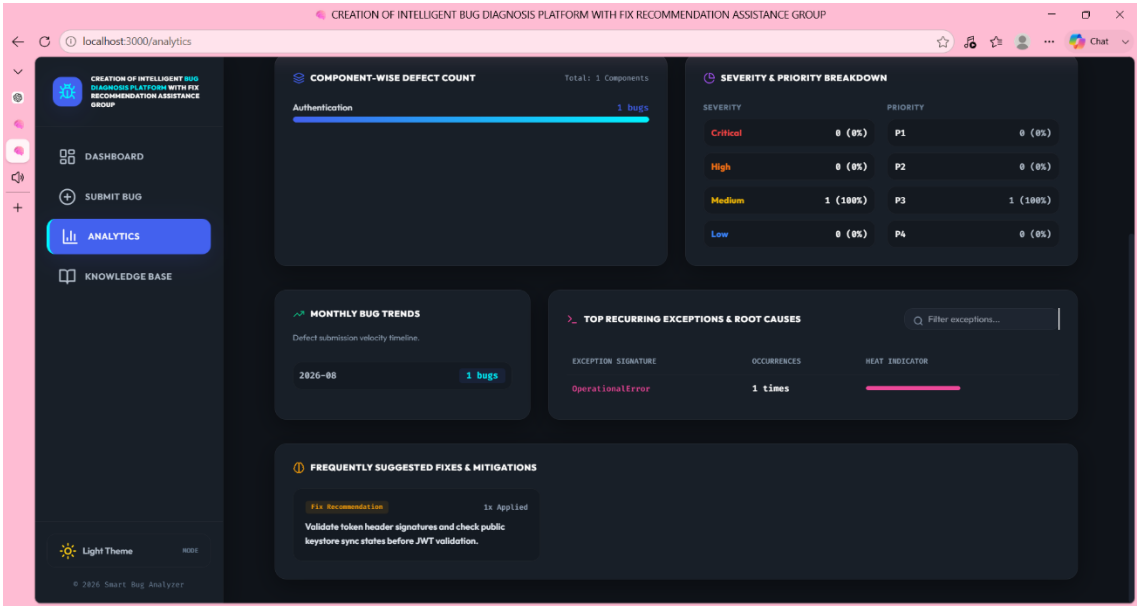
7.Root-Cause Analysis



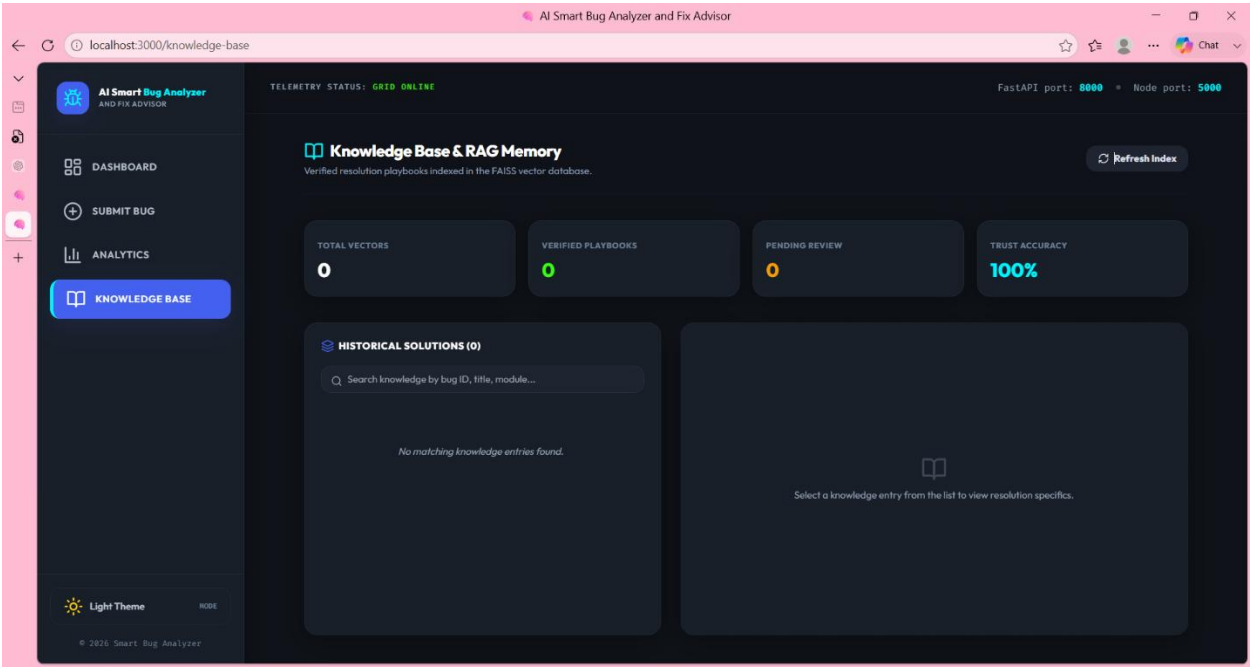
8.Fix Recommendation



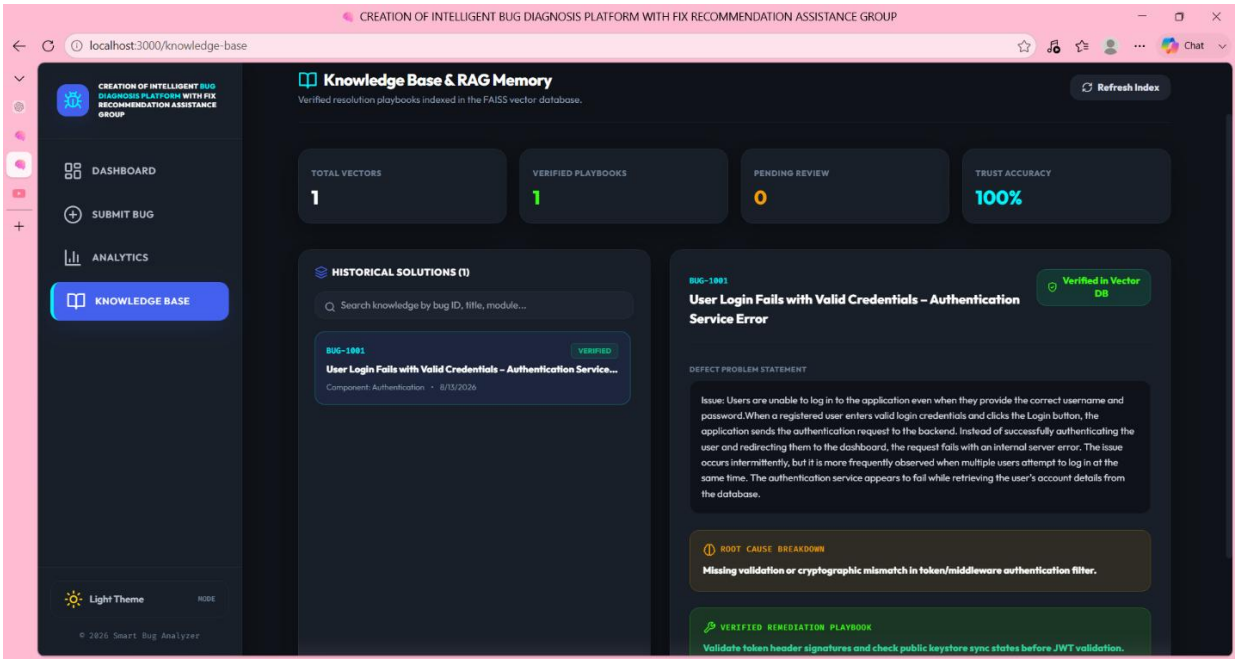
9.Defect Analytics Dashboard



10.Knowledge Base Update



11.Final Demonstration



REFERENCES

1. React.js Documentation
2. Fast API Documentation
3. Lang Chain Documentation
4. Lang Graph Documentation
5. FAISS Documentation
6. Sentence Transformers Documentation
7. PostgreSQL Documentation
8. Gemini API Documentation
9. Research literature on Retrieval-Augmented Generation
10. Research literature on semantic similarity and vector databases
11. Research literature on automated software defect analysis